Laboratorium Tasks

by Paweł Paduch (https://achilles.tu.kielce.pl/portal/author/332bd2d158a24964b2b98a7fc2879842) published 3 years ago, last modified 3 years ago

Pierwsze laboratorium z Taskami w C#

1. **Pierwsze Taski [0,5 pkt.]**
   - **Utworzyć solucję z projektem FirstTasks**
   - **Przepisać (przekleić) poniższy kod:**

```
using System;
using System.Collections.Generic;
using System.Threading;
using System.Threading.Tasks;


namespace FirstTasks
{
    class Program
    {
        private const int iterNo = 10;
        static void Main(string[] args)
        {
            Task t1 = new Task(() =>
            {
                Random rand = new Random(Task.CurrentId.Value);//init random generat
                Console.WriteLine($"Thread id {Thread.CurrentThread.ManagedThreadId}
                Thread.Sleep(rand.Next(0, 100));
                for (int i = 0; i < iterNo; i++)
                {
                    Console.Write($"#{i}# ");
                    Thread.Sleep(rand.Next(500));
                }
                Console.WriteLine($"\nTask {Task.CurrentId.Value} finished");
            });
            t1.Start();

            Console.WriteLine($"Thread id {Thread.CurrentThread.ManagedThreadId}");
            Random rand = new Random();
            Thread.Sleep(rand.Next(0, 100));
            for (int i = 0; i < iterNo; i++)
            {
                Console.Write($"{{{i}}} ");
                Thread.Sleep(rand.Next(500));
            }
            Console.WriteLine("\nMain task finished");
            }
    }
}
```

   - **Task 1 i Main Task wypisują na ekran liczby w losowych odstępach czasu. Na koniec każdy z tasków wypisuje komunikat o zakończonej pracy. Niestety nie zawsze komunikat z Task'a 1 jest**

widoczny. Znajdź rozwiązanie tego problemu.

2. **Task tworzony z Akcji [0,5 pkt.]**
   - Do powyższego programu dopisać funkcję `static void task2()` realizującą to samo zadanie co task *t1* z tą różnicą, że numer iteracji będzie obramowany nawiasami kwadratowymi. Przykładowo [0],[1],[2]...itd..
   - Task *t1* był tworzony za pomocą notacji lambda (anonimowo). Utworzyć task *t2* realizujący akcję `void task2()`.
   - Wystartować *t2* zaraz po utworzeniu.
   - Poczekać na *t2* tuż za *t1*.

3. **Przekazanie parametru [0,5 pkt.]**
   - Bazując na akcji *task2* utworzyć akcję `static void task3(object o)`
   - Wewnątrz akcji *task3* zrzutować object o do zmiennej typu *int* i użyć jej zamiast *iterNo* (liczby iteracji w pętli)
   - Akcja *task3* ma wyświetlać liczby iteracji w nawiasach okrągłych czyli (0),(1),(2),...itd.
   - Utworzyć task *t3* używając akcji *task3* przekazując liczbę iteracji przez parametr.
   - Wystartować t3 zaraz po t2
   - Poczekać na t3 zaraz po t2

4. **Przekazanie krotki (tuple) [0,5 pkt.]**
   - Do programu dodać poniższą ację

```
static void task4(object arg)
       {
           (int id, string name) tupleArg = ((int id, string name))arg; //cast tupl
           string taskName = tupleArg.name;
           int taskId = tupleArg.id;
           Console.WriteLine($"I'm {taskName} and have ID: {taskId}");
       }
```

   - Utworzyć i wystartować task w poniższy sposób

```
(int id, string name) idName = (4, "Task4"); //tupple
Task t4 = new Task(task4,idName);
t4.Start();
```

   - Task 4 wykorzystuje krotkę (zmienną z wieloma wartościami). Jest to szybki sposób na przekazanie kilku parametrów bez konieczności definiowania klasy i tworzenia jej obiektu.
   - Krotkę można też wprost przekazać do taska:

```
Task t4b = new Task(task4,(4, "Task4b"));
t4b.Start();
```

   - Dodać oczekiwanie na *t4* zaraz po oczekiwaniu na *t3*

5. **Przekazanie obiektu zdefiniowanej klasy [0,5 pkt.]**
   - Utworzyć klasę *MyParam*

```
public class MyParam
   {
       public string Name { get; set; }
       public int Id { get; set; }
   }
```

- Utworzyć akcję *task5* wykonującą to samo co *task4* ale pobierającą dane z przekazanego obiektu (pamiętać o rzutowaniu do klasy *MyParam)*.
- Utworzyć obiekt *myParam* klasy *MyParam* zainicjować pola *Name* i *Id*
- Utworzyć task *t5* wykonujący akcję *task5*, przekazać obiekt *myParam*
- Wystartować task *t5*
- Dodać oczekiwanie na *t5* po oczekiwaniu na *t4*

6. **Utworzenie listy tasków [0,5 pkt.]**
   - Zadeklarować i ustawić zmienną `int taskNo = 5;` Będzie to liczba tasków do uruchomienia.
   - Utworzyć listę `List<Task> t5list = new List<Task>();`
   - W pętli utworzyć i dodać taskNo tasków wykonujących akcję task5. Każdy task powinien posiadać id zgodne z numerem iteracyjnym pętli oraz nazwę w postaci "Task" + i.*ToString*()
   - Taski utworzone startować zaraz po dodaniu do listy
   - Można użyć `Task.Factory.StartNew` by zwrócić utworzony i wystartowany task.
   - Dodać czekanie na wszystkie taski (pętla foreach) po czekaniu na *t5*.

7. **Projekt Snow [0,5 pkt.]**
   - Dodać nowy projekt *Snow*. przekopiować poniższy kod metody *Star()*.

```
static void Star(object o)
        {
            int height = 26;
            int width = 79;
            int delay = 200;
            int orgx;
            int orgy;
            ConsoleColor orgcolor;
            int prevx;
            int currentx;
            int prevy = 0;
            int currenty=0;
            int iterations = 5; //how many times sars wrap the screen verticaly
            ConsoleColor? color = null; //if nul then random
            Random rand = new Random((int)DateTime.Now.Ticks);

//  Console.WriteLine($"Task started on thread Id {Thread.CurrentThread.ManagedThrea
            currentx = prevx = rand.Next(width - 1);
            Console.CursorVisible = false; //we don't need blinking cursor
            for (int y = 0; y < height*iterations; y++)
            {
                currenty = y % height; //just from interation
                currentx = prevx + rand.Next(-1,1);
                if (currentx < 0) currentx = width;
                if (currentx > width) currentx = 0;
                lock (Console.Out)
                {
                    //remember orginal position and color
                    orgx = Console.CursorLeft;
                    orgy = Console.CursorTop;
                    orgcolor = Console.ForegroundColor;
                    //erease elier star
                    Console.SetCursorPosition(prevx, prevy);
                    Console.Write(' ');
                    //set color and new position
                    if (color == null) Console.ForegroundColor = (ConsoleColor)rand.
                    else Console.ForegroundColor = color.Value;
                    Console.SetCursorPosition(currentx, currenty);
                    //put the star
                    Console.Write('*');
                    //restore position and color
                    Console.SetCursorPosition(orgx, orgy);
                    Console.ForegroundColor = orgcolor;
                }
                prevx = currentx;
                prevy = currenty;
                Thread.Sleep(delay);
            }
        }
```

Jest to Akcja wyświetlająca spadającą kolorową gwiazdkę. Gwiazdka po dojściu do krawędzi konsoli pojawia się po drugiej stronie.

- **Utworzyć listę 50 wystartowanych tasków z akcją Star. Obiekt przekazywany na razie może być null.**

- Zaczekać na wszystkie taski za pomocą metody *Task.WaitAll*

8. **Więcej śniegu. [0,5 pkt]**
   - Jeżeli płatki śniegu nie pojawiają się wszystkie od razu, odkomentować linię `Console.WriteLine($"Task started on thread Id {Thread.CurrentThread.ManagedThreadId}");` i zobaczyć ile tasków uruchamia się jednocześnie.
   - Zmienić liczbę wątków w puli. Przed startem dodać linię `ThreadPool.SetMinThreads(64, 64);`
   - Jeżeli płatki pojawiają się lawinowo. Dodać losowe opóźnienie od 0 do 100ms przed każdym utworzeniem taska.

9. **Przekazanie informacji do akcji Star [0,5 pkt.]**
   - Utworzyć klasę *StarParameter* zawierającą takie zmienne jak:
     - *Random rand*
     - *int iterations*
     - *ConsoleColor? color*
   - Przed utworzeniem taska tworzyć obiekt klasy *StarParameter* z zadaną liczbą iteracji, losowym kolorem i przekazaną referencją do obiektu klasy *Random*.
   - Przekazać obiekty do tasków a w nich zrzutować i odczytać dane.

10. **Dodatkowe działania [0,5 pkt.]**
    - Zabezpieczyć *random* by był threadsafe. (w .net 6 wprowadzono *Rand.Shared,* nie wymaga tworzenia instancji i jest z założenia thread safe)
    - Akcja *Star* powinna odczytywać bieżący rozmiar konsoli i dopasowywać wyliczenia tak by śnieg padał na całym ekranie, nawet gdy okno powiększymy.